Getting to know RStudio - Notes

Welcome!

Welcome to Stat 133! The system that you are currently working in is called RStudio and it is an environment in which to use the R programming language. You will be using R and RStudio throughout this course to analyze real data and come to informed conclusions. To straighten out which is which: R is the name of the programming language itself, and RStudio is a convenient interface for using R.

As the course progresses, you are encouraged to explore beyond what we discuss; a **willingness to experiment** will make you a much better scientist and researcher (and programmer!). Before we get to that stage, however, you need to build some competence in R. Just like learning any spoken language, we need to learn some of the “vocabulary” and “syntax” associated to the R language. We will begin with some of the fundamental building blocks of R and Rstudio: the interface, data types, variables, importing data, and plotting data. We will then move into performing statistical tests, and you will even perform your own statistical study!

R is widely used by the scientific community as a no-cost alternative to expensive commercial software packages like SPSS and MATLAB. It is both a statistical software analysis system and a programming environment for developing scientific applications. Scientists routinely make available for free R programs they have developed that might be of use to others. Thousands of packages can be downloaded for all types of scientific computing applications.

Exploring RStudio

We will begin by looking at the RStudio software interface. Each section of the interface is called a “pane” and each pane has different features and capabilities. There are four main panes.

- The pane in the bottom left is the **R Command Console**, this is where you can type R commands for immediate execution.

- The pane in the upper left portion of the window, in which you are currently reading this file, is an area for editing R source code for scripts and functions and for viewing R data frame objects. New tabs will be added as new R code files and data objects are opened.
- The pane in the upper right portion of the window is an area for browsing the variables in the R workspace **environment** and the R command line **history**.
- The pane in the lower right portion of the window has several tabs. The **Files** tab is an area for browsing the files in the current working directory. The **Plot** tab is for viewing graphics produced using R commands. The **Packages** tab lists the R packages available. Other packages can be loaded. The **Help** tab provides access to the R documentation. The **Viewer** tab is for viewing local web content in the temporary session directory (not files on the web).

Let's explore some of these panes in more detail.

Bottom Left Pane

Let's begin with the **Console**, in the bottom left pane. This is where you type R commands for immediate execution. Click in the Command Console next to the `>` symbol. You should see a blinking cursor that tells you the console is the current focus of keyboard input. Type `5+5` in the console and press enter. The `[1]` indicates that the line begins with the first (and only) element of the result, which is the number 10.

You can also execute R's built-in functions (or functions you create). Type the following command into the console and press enter: `exp(pi)`. In R, `pi` is a special constant to represent the number, and `exp()` is the exponential function. As before, the `[1]` tells you that the first (and only) element of the result is the number $e^{\pi} = 23.14069$.

Bottom Right Pane

There are a number of helpful tabs in the bottom right pane.

Files

If you click on the files tab, you should see all the files associated to this project. If you were running actual RStudio rather than RStudio online, you would see actual files that are on your computer. Go ahead and click on the document called "GettingToKnowRStudio-Practice.qmd" which should open it in a new tab in the top left pane. To navigate back to the notes, select the name from the top of the top left pane.

Packages

The **Packages** tab can be used to install special sets of code called "packages". Let's try installing a package. Under the **Packages** tab, click the **Install** button (next to the blue arrow). Once there, type in `ggplot2` where it says "Packages", then press **Install**. You might see a lot of text showing up in the bottom left pane; this is RStudio downloading and installing the `ggplot2` package.

Help

The **Help** tab is useful if you want to know more about an object built in to the R programming language. An easy way to access a particular help file is to actually type into the console in the bottom left pane. In the console, type `help(exp)` and then press enter. You should see a file appear in the **Help** tab in the bottom right pane telling you what the `exp()` function does. An equivalent way to access the help pages is by typing "?" followed by the function name and then pressing enter (for example `?exp`).

Make liberal use of the help documentation! Nearly any function or object in base R or from an R package will have documentation written for it, so when in doubt, check the documentation.

Top Right Pane

Next we will look at two parts of the top right pane.

History

Click on the **History** tab in the top right pane. You should see a list of all of the commands that you have run in this project so far.

Environment

The **Environment** tab keeps track of all of the names and other information of any variables that you have created during your R session that are available for use in other commands.

Currently, the environment should be empty, so let's add something to it. In the console in the bottom left pane, type the following three lines of code, pressing enter between each line:

```
x <- 29.325
y <- log(a)
z <- a/b
```

Now take a look at the Environment pane. The variables `x`, `y`, and `z` are now a part of your R work space. You can reuse these variables as part of other command. Try typing the following in the console:

```
v <- c(x, y, z)
v
```

The variable `v` is a vector created using the "concatenate" function `c()`. This function combines its inputs into a vector or list. Look at the environment tab where it lists the variable `v`. The text `num [1:3]` tells us that the variable `v` is a vector with elements `v[1]`, `v[2]`, and `v[3]`. In the console try typing `v[1]`, which displays the first object in the `v` vector. Does this number look familiar?

Top Left Pane

Writing code

As we have seen already, the top left pane is where you can see and interact with the files that you have opened. Many of the files we will work with this semester will include **Executable Code Blocks**. These are sections of a file in which you can write and run R code, similar to how you write and run code in the console.

Below is an example of an executable code block. You can see that in the code block, I have written two lines of code (notice that they go below the line that says `{r}`). Press the small green arrow to run the code in the code block.

```r
exp(log(3))
```

```
[1] 3
```

```r
2+3
```

```
[1] 5
```

Note that this does the same thing as typing this code into the console and pressing enter. When you run a line or block of code, it's simply "sending" that code to the console. Run the code again and watch the console this time - you'll see the code in the code block show up in the console.

You can also run individual lines of code (rather than running all of the code in a block). The easiest way to do this is by pressing Ctrl/Cmd + Enter - this will run the line your cursor is on.

Displaying data frames

Another feature of the top left pane is that you can display data from "data frame" objects. We will use a data set that is built into R as an example. Run the code in the code block below. `mtcars` is a sample data frame built into R, and we are saving it under the variable name `df`.

```
df <- mtcars
```

You should now see the variable `df` in the environment pane in the top right corner. If you click on the name `df` in the environment pane, it should pull up the data in a new tab in this top left pane. This is for viewing the data only; the data cannot be edited.

You can also list the data in the console or in a code block by typing the name of the data frame object. Run the following code block to see the data displayed another way.

```
df
```

```
                     mpg cyl  disp  hp drat    wt  qsec vs am gear carb
Mazda RX4           21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag       21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
Datsun 710          22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive      21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout   18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
Valiant             18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
Duster 360          14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
Merc 240D           24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
Merc 230            22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
Merc 280            19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
Merc 280C           17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
Merc 450SE          16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
Merc 450SL          17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
Merc 450SLC         15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
Cadillac Fleetwood  10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
Chrysler Imperial   14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
Fiat 128            32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
Honda Civic         30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
Toyota Corolla      33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
Toyota Corona       21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
Dodge Challenger    15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
AMC Javelin         15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
Camaro Z28          13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
```

```
Pontiac Firebird    19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
Fiat X1-9           27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
Porsche 914-2       26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
Lotus Europa        30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
Ford Pantera L      15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
Ferrari Dino        19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
Maserati Bora       15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
Volvo 142E          21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
```

We will get lots of practice with data frames in the coming weeks.

Types of R files

At it's most basic, we can simply type our code directly into the console and run it. However, while that works, we can't easily save the code we've entered into the console - it'd be nicer if we could write the code in a file so that we can save it.

The simplest type of file for running R code is an R script, which has the file extension ".R" (e.g. "myscript.R") and is simply a text file that contains R code. While R files are frequently used in practice, we won't be using them in this class.

Instead we'll be using what are called Quarto documents, which is what this file is - these have the file extension ".qmd". In an R file, we assume that everything contained in the file is R code, and if we want to include chunks of text like this, we have to use comments to indicate that something isn't R code (you'll see what comments look like in the practice file). In a Quarto file, this is the opposite - everything in the file is assumed to be text unless you specifically create a code chunk. This makes it a good fit for educational settings (like this class) since we can have large sections of explanation followed by runnable code chunks for demonstration.

Another important feature of Quarto files is that they can be rendered into different file formats, like PDFs and Word Documents. When rendering, the code in the code chunks is run and the output will be shown in the resulting output document. This makes Quarto documents convenient for things like writing reports where we use R to generate figures - the code to generate the figures can be embedded into the document itself, so we don't have to worry about saving out the figures and then putting the results into a Word document.

One more type of file you may come across is R Markdown files. This is the predecessor to Quarto documents and is very similar - we won't use them in this class, but you may run across them from time to time.

More Practice with RStudio

For more practice with RStudio, and to practice submitting files in Canvas, please open the "GettingToKnowRStudio-Practice.qmd" file again. There will likely be a tab already open at the top of this pane. Once that file is open, follow the instructions therein.